Centro Universitário de Excelência
Sistemas de Informação

Construindo a Web API da MecâniQA - Fase 3

Autores: Anthony Kevin Missias
Guilherme Brandão de Araújo
Icaro Menezes dos Santos Vinhas
Igor Santos de Britto
João Vitor Souza Bomfim

Itabuna, 2026

Agenda

O objetivo desta apresentação é demonstrar a evolução arquitetural e a implementação da persistência definitiva em banco de dados na Web API da MecâniQA.

1. **Contextualização:** A transição da persistência em memória para o banco de dados relacional e as restrições técnicas da Fase

2. **Conectividade JDBC e Modelagem SQL:** A classe `DatabaseConnection`, o uso de `PreparedStatement` e o versionamento do script MySQL.

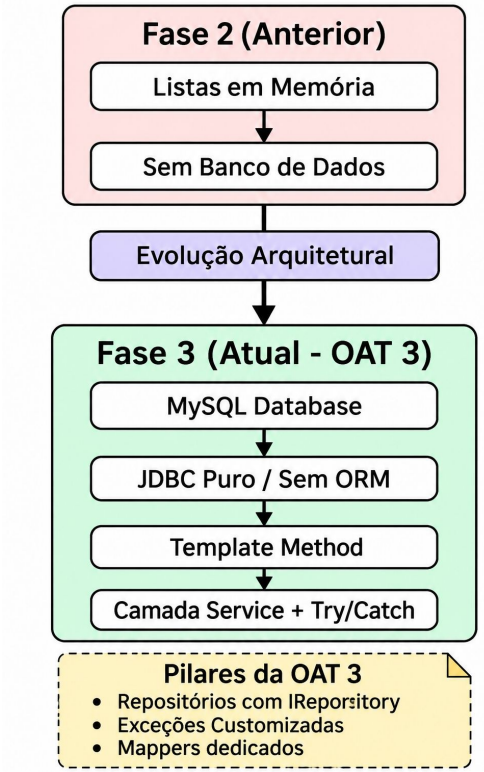
3. **Padronização e Template Method:** O contrato genérico IRepository<T, ID> e a eliminação de repetição de código com AbstractJdbcRepository.

4. **Mapeamento Relacional e Transações:** As classes utilitárias ResultSetMapper e o controle transacional na entidade associativ

5. **Camada de Service e Exceções Customizadas** A Regra de Ouro da arquitetura e o tratamento manual de erros via try/catch sem @ExceptionHandler

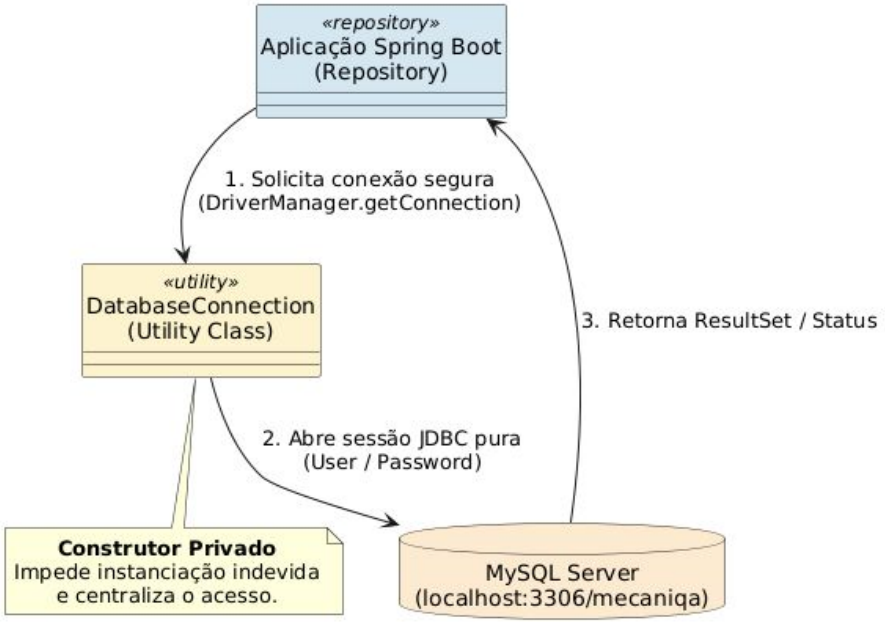
Contextualização do Problema

- **O Desafio:** Superação do armazenamento volátil em listas estáticas em memória, que descartava todos os dados ao reiniciar o servidor.
- **O Objetivo:** Transição para persistência definitiva e segura no MySQL, construindo a camada de acesso a dados manualmente com JDBC puro e sem abstrações de ORM (como JPA/Hibernate).
- **Evolução Arquitetural:** Introdução da camada @Service para isolamento de regras de negócio, padronização da execução via padrão Template Method e tratamento robusto de falhas com Exceções Customizadas.



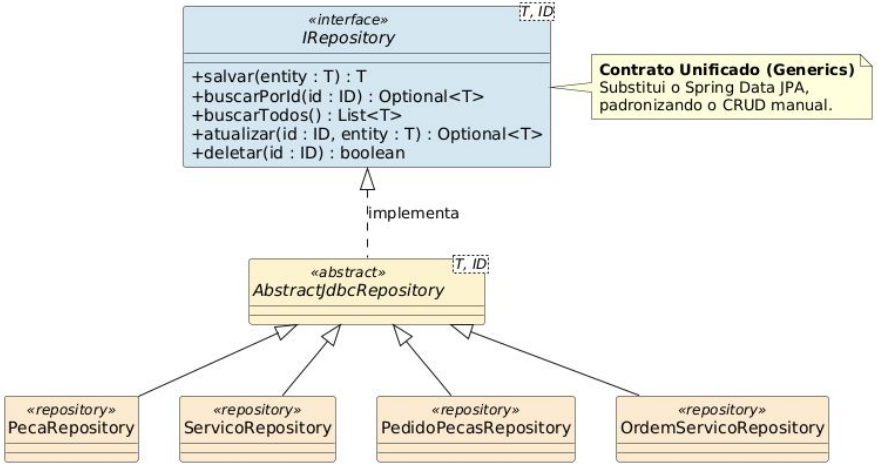
Conexão JDBC e Script SQL

Banco MySQL estruturado pelo `script_mecaniqa.sql` (DDL/DML na raiz), com conexões centralizadas pela `DatabaseConnection` (construtor privado via `DriverManager`) e consultas seguras via `PreparedStatement` parametrizado em constantes.



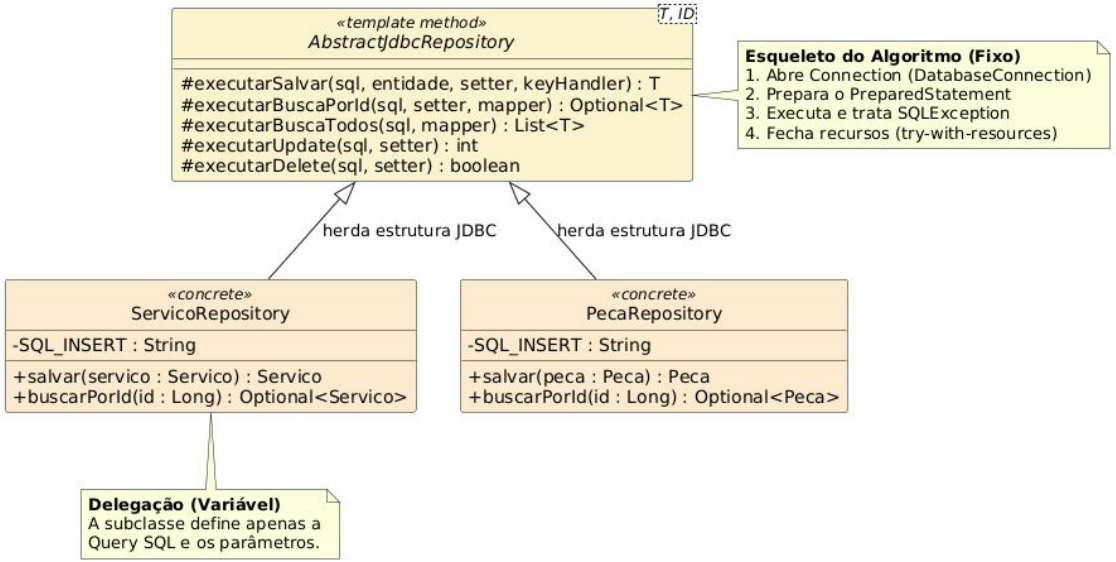
Padronização de Repositórios

- **Restrição de Spring Data:** Como o uso de interfaces automatizadas do Spring Data JPA é proibido nesta etapa, criamos nossa própria interface genérica
- **Uso de Generics (`<T, ID>`):** `T` representa a Entidade de domínio (ex: `Peca`, `Servico`, `OrdemServico`) e `ID` representa o tipo da chave primária (`Long`).
- **Contrato Unificado:** Obriga todos os repositórios do sistema a seguirem a mesma assinatura para as 5 operações fundamentais do CRUD (`salvar`, `buscarPorId`, `buscarTodos`, `atualizar` e `deletar`).



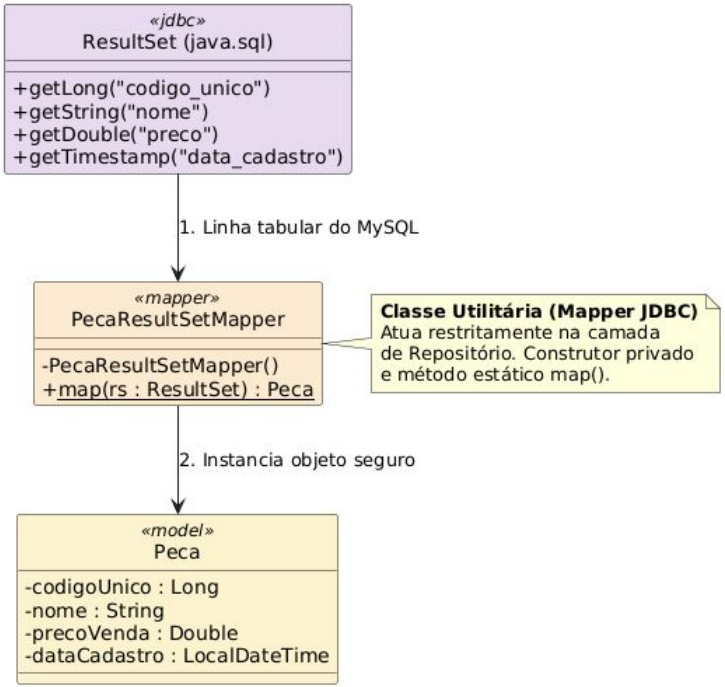
Template Method

- **Banco e Script:** MySQL oficial com DDL e DML centralizados no `script_mecaniqa.sql` na raiz do projeto.
- **Conexão:** Classe `DatabaseConnection` com construtor privado, centralizando instâncias via `DriverManager.getConnection()`.
- **Segurança:** Consultas parametrizadas (`?`) em constantes via `PreparedStatement`, garantindo proteção contra SQL Injection e ganho de performance.



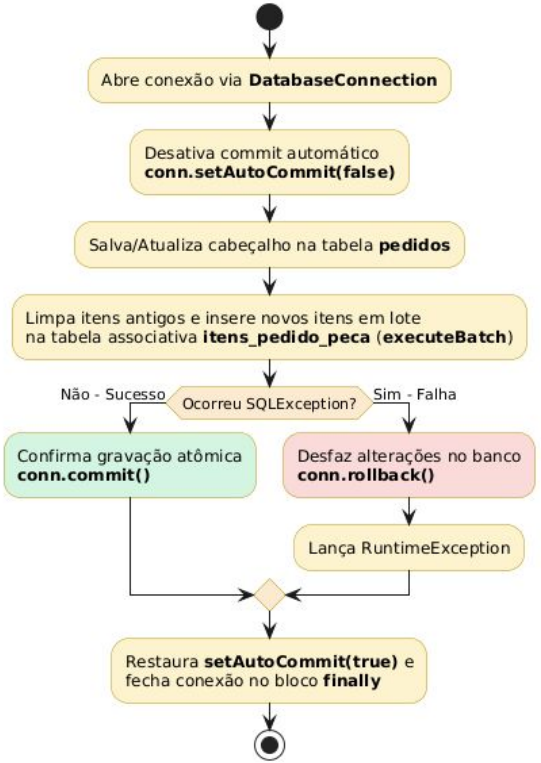
Mapeadores de ResultSet

- **Impedância Objeto-Relacional:** O JDBC puro retorna dados tabulares via `ResultSet`, formato incompatível com a manipulação direta nas regras de negócio.
- **Solução (`ResultSetMapper`):** Classes utilitárias no pacote `mapper`.JDBC (construtor privado e método estático `map`) exclusivas para converter registros do `ResultSet` em entidades de domínio.
- **Isolamento de Camadas:** Os mappers atuam restritos à camada de Repositório, blindando Service e Controller contra dependências de `java.sql.ResultSet`



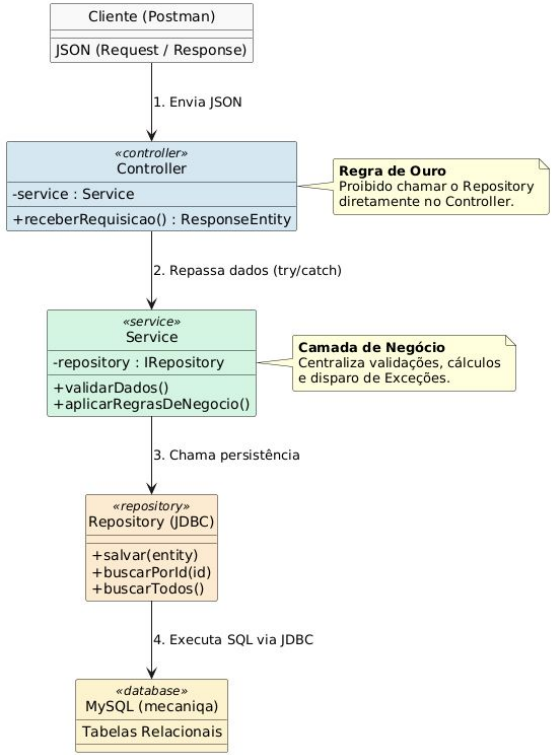
Transações e Associativas no Banco

- **Atomicidade:** Gravação coordenada entre o cabeçalho (`pedidos`) e seus itens na tabela associativa (`itens_pedido_peca`).
- **Controle Transacional Manual:** Desativação do autocommit (`setAutoCommit(false)`), confirmando a transação via `commit()` apenas no sucesso integral e aplicando `rollback()` no bloco `catch` em caso de erro.
- **Processamento em Lote e Modelagem:** Inserção otimizada dos itens via batch (`addBatch()` e `executeBatch()`), com a tabela `itens_pedido_peca` sem chave `UNIQUE` composta para viabilizar múltiplos lançamentos da mesma peça.



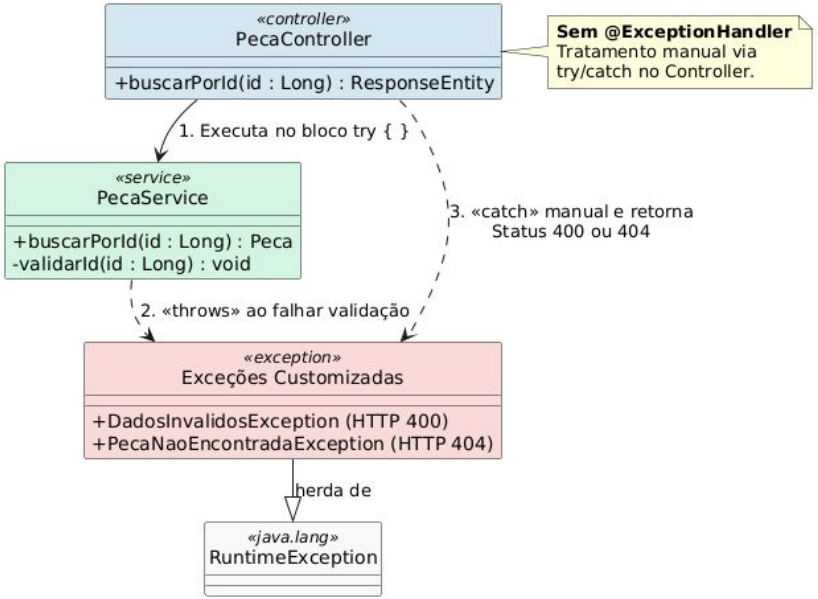
Camada de Service e Regra de Ouro

- **Regra de Ouro:** Controllers são estritamente proibidos de chamar Repositories JDBC diretamente.
- **Papel do Service:** Classes `@Service` concentram todas as validações, regras de negócio, cálculos e orquestração de persistência com os repositórios.
- **Desacoplamento:** Controllers funcionam exclusivamente como adaptadores de entrada HTTP, delegando a execução ao Service e retornando o `ResponseEntity` com o status HTTP correspondente.



Exceções Customizadas e try/catch

- **Regra de Ouro:** Controllers são estritamente proibidos de chamar Repositories JDBC diretamente.
- **Papel do Service:** Classes `@Service` concentram todas as validações, regras de negócio, cálculos e orquestração de persistência com os repositórios.
- **Desacoplamento:** Controllers funcionam exclusivamente como adaptadores de entrada HTTP, delegando a execução ao Service e retornando o `ResponseEntity` com o status HTTP correspondente.



Considerações finais

- **Pontos de interesse**
  - A construção manual da persistência via **JDBC puro** permitiu compreender na prática o ciclo real de comunicação com o banco de dados **MySQL** antes da adoção de frameworks ORM automatizados.
  - A combinação da interface genérica **IRepository** com o padrão comportamental **Template Method** (AbstractJdbcRepository) e os **ResultSetMappers** eliminou código repetitivo e garantiu o fechamento seguro das conexões.
  - A introdução da camada **Service** aliada às **Exceções Customizadas** e ao tratamento manual com try/catch consolidou a separação de responsabilidades na arquitetura da MecâniQA

Referências

- **Referências do projeto**
  - **Conteúdos e Materiais Didáticos:**

    Conteúdos apresentados nas aulas teóricas e Sessões Tutoriais (ST) da Unex.

  - **APIs e Controllers:**

    SPRING BOOT. Spring Boot Reference Documentation. 2026. Disponível em: https://docs.spring.io/spring-boot/documentation.html